Homework 2

sli602

October 2026

All the codes are provided in github repo: https://github.com/SiyuLi915/CS521

1 Problem 1

(a) The exact sum is

$$I_1 + I_2 = [a_1 + a_2, b_1 + b_2] + i[c_1 + c_2, d_1 + d_2],$$

(b) Using real-interval arithmetic,

$$\begin{aligned}[u, v][s, t] &= [\min\{us, ut, vs, vt\}, \max\{us, ut, vs, vt\}],\\ [u, v] - [s, t] &= [u - t, v - s].\end{aligned}$$

The smallest complex interval containing $I_1 I_2$ is

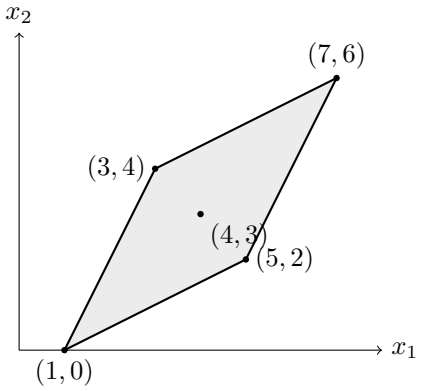
$$\begin{aligned}J = &\big([a_1, b_1][a_2, b_2] - [c_1, d_1][c_2, d_2]\big)\\ &+ i\big([a_1, b_1][c_2, d_2] + [c_1, d_1][a_2, b_2]\big).\end{aligned}$$

2 Problem 2

(a) All noise symbols range independently over $[-1, 1]$. Taking $(\varepsilon_1, \varepsilon_2) \in \{-1, 1\}^2$ gives

$$\phi = \text{conv}\{(1, 0), (5, 2), (7, 6), (3, 4)\},$$

a parallelogram centered at $(4, 3)$.



(b) Yes. The exact affine transformer gives

$$\hat{x}_3 = 10 + 4\varepsilon_1 + 5\varepsilon_2 \in [1, 19],$$
$$\hat{x}_4 = -1 - \varepsilon_1 + \varepsilon_2 \in [-3, 1].$$

Since $\hat{x}_3 > 0$, set $\hat{x}_5 = \hat{x}_3$. For the other ReLU, introduce a fresh $\varepsilon_3 \in [-1, 1]$ and use

$$\hat{x}_6 = \frac{1}{4}\hat{x}_4 + \frac{3}{8} + \frac{3}{8}\varepsilon_3 = \frac{1}{8} - \frac{1}{4}\varepsilon_1 + \frac{1}{4}\varepsilon_2 + \frac{3}{8}\varepsilon_3.$$

This is sound because

$$0 \le \text{ReLU}(t) - \frac{1}{4}t \le \frac{3}{4} \qquad (t \in [-3, 1]).$$

Therefore,

$$\begin{aligned}\hat{x}_5 - \hat{x}_6 &= \frac{79}{8} + \frac{17}{4}\varepsilon_1 + \frac{19}{4}\varepsilon_2 - \frac{3}{8}\varepsilon_3 \\ &\ge \frac{79}{8} - \frac{17}{4} - \frac{19}{4} - \frac{3}{8} = \frac{1}{2} > 0.\end{aligned}$$

By soundness, $x_5 \ge x_6$ for every input in ϕ.

3 Problem 3

(a) An exact MILP encoding of $y = \max(x_1, x_2)$ is

$$\begin{aligned}
&a_i \le x_i \le b_i \quad (i = 1, 2), \qquad z \in \{0, 1\},\\
&y \ge x_1, \qquad y \ge x_2,\\
&y \le x_1 + \max\{0, b_2 - a_1\}(1 - z),\\
&y \le x_2 + \max\{0, b_1 - a_2\}z.
\end{aligned}$$

When $z = 1$, the constraints force $y = x_1 \ge x_2$; when $z = 0$, they force $y = x_2 \ge x_1$. The constants make the unused upper bound nonrestrictive, so every valid maximum, including a tie, is represented.

(b) Since $x_3 - x_4 = x_2 + 2 > 0$, the first maxpool simplifies to $x_7 = x_3$. Using $x_5 \in [-1, 1]$ and $x_6 \in [0, 1]$, the network encoding is

$$\begin{aligned}
&0 \le x_1, x_2 \le 1, \qquad z \in \{0, 1\},\\
&x_3 = x_1 + x_2, \qquad x_4 = x_1 - 2,\\
&x_5 = x_1 - x_2, \qquad x_6 = x_2, \qquad x_7 = x_3,\\
&x_8 \ge x_5, \qquad x_8 \ge x_6,\\
&x_8 \le x_5 + 2(1 - z), \qquad x_8 \le x_6 + z,\\
&x_9 = x_7, \qquad x_{10} = -x_7 + x_8 - \frac{1}{2}.
\end{aligned}$$

The maxpool inequalities also encode $x_8 - x_6 = \mathrm{ReLU}(x_5 - x_6)$.

Under these constraints,

$$\begin{aligned}
x_9 - x_{10} &= 2(x_1 + x_2) - x_8 + \frac{1}{2}\\
&= \begin{cases} x_1 + 3x_2 + \frac{1}{2}, & z = 1,\\ 2x_1 + x_2 + \frac{1}{2}, & z = 0. \end{cases}
\end{aligned}$$

Both cases are at least $\frac{1}{2}$, attained at $(x_1, x_2) = (0, 0)$. Thus

$$\min(x_9 - x_{10}) = \frac{1}{2} > 0,$$

so MILP analysis proves $x_9 > x_{10}$ throughout $[0, 1]^2$.

4 Problem 4

Colab's link: https://colab.research.google.com/drive/1cMU6wZuTUsxx7Q7bxLA0pBMB4s8Re6mw?usp=sharing

5 Problem 5

(a) First, I kept DeepPoly's shape and changed only the two transformers (`mycert.cf`) but the certifier is less precise than DeepPoly, so I changed the shape to make it better. My final design is the following:

Shape. I add a second polyhedral lower bound, `La`:

```
(Float l, Float u, PolyExp L, PolyExp U, PolyExp La)
```

with the extra constraint `curr[La] <= curr`.

ReLU. When the input crosses zero, both 0 and x are sound lower bounds. `L` keeps DeepPoly's area-heuristic choice, and `La` stores the other choice. Stable neurons use the same bound in `L` and `La`. The remaining ReLU computations are unchanged.

Affine. I backsubstitute twice, once using `L` and once using `La` for nonnegative coefficients; both passes use `U` for negative coefficients. I set

$$l = \max\{l_L, l_{L^a}\}.$$

Both results are sound lower bounds, so their maximum is sound and no looser than the `L`-based pass on the same stored state. The computation of `u` is unchanged, while `L`, `U`, and `La` are all set to the exact affine expression.

Why useful. DeepPoly must commit to one lower bound (0 or x) for each unstable ReLU, and the area heuristic only looks at that one neuron. The choice that gives the tightest bound after backsubstitution can be different. `deeppoly2l` keeps both choices and, at each Affine neuron, uses the larger of the two lower bounds. Given the same polyhedral bounds from the earlier layers, this lower bound is never looser than DeepPoly's. The cost is one additional backsubstitution pass for each Affine layer.

(b) Output:

```
Proved Affine
Proved Relu
Provesound result: None
```

(c) Experiments

	DeepPoly		`deeppoly2l`	
ϵ	Verified	Time/image (s)	Verified	Time/image (s)
0.005	99%	0.213	99%	0.446
0.01	94%	0.273	94%	0.446
0.02	83%	0.275	83%	0.442
0.03	49%	0.287	49%	0.366
0.05	2%	0.319	2%	0.363

Comparison. The two certifiers verify the same percentage of images for every ϵ. `deeppoly2l` takes about 1.1–2.1× as long for each image, because each Affine layer has one more backsubstitution pass.

Why the results are the same. I compared the final bounds of both certifiers on each image. The new certifier does tighten some bounds, but the gain is very small. The largest increase of a bound on $\text{logit}_y - \text{logit}_j$ is 2.3×10^{-3} at $\epsilon = 0.02$ and 7.5×10^{-5} at $\epsilon = 0.03$. In comparison, the closest unverified image at $\epsilon = 0.02$ has a margin of -0.071. So no result changes.

Code. `https://github.com/SiyuLi915/CS521/tree/main/hw2/q5/deeppoly2l.cf`

(d)

```
func slope_hi(Neuron n) = (1.0 - n[l]) / (n[u] - n[l]);
func ht_lower_hi(Neuron n) = (slope_hi(n) * n) + (n[l] - (slope_hi(n) * n[l]));
func ht_upper_hi(Neuron n) = n[l] + n[u] <= 2.0 ? n : 1.0;
func slope_lo(Neuron n) = (n[u] + 1.0) / (n[u] - n[l]);
func ht_lower_lo(Neuron n) = n[l] + n[u] >= (0.0 - 2.0) ? n : (0.0 - 1.0);
func ht_upper_lo(Neuron n) = (slope_lo(n) * n) - ((slope_lo(n) * n[l]) + 1.0);
func ht_lower_both(Neuron n) = n[l] + n[u] >= 0.0 ?
    ((2.0 / (n[u] + 1.0)) * n) + ((2.0 / (n[u] + 1.0)) - 1.0) : (0.0 - 1.0);
func ht_upper_both(Neuron n) = n[l] + n[u] <= 0.0 ?
    ((2.0 / (1.0 - n[l])) * n) + (1.0 - (2.0 / (1.0 - n[l]))) : 1.0;

HardTanh -> (prev[u] <= (0.0 - 1.0)) ?
  ((0.0 - 1.0), (0.0 - 1.0), (0.0 - 1.0), (0.0 - 1.0)) :
  ((prev[l] >= 1.0) ? (1.0, 1.0, 1.0, 1.0) :
  (((prev[l] >= (0.0 - 1.0)) and (prev[u] <= 1.0)) ?
      (prev[l], prev[u], prev, prev) :
  ((prev[l] >= (0.0 - 1.0)) ?
      (prev[l], 1.0, ht_lower_hi(prev), ht_upper_hi(prev)) :
  ((prev[u] <= 1.0) ?
      ((0.0 - 1.0), prev[u], ht_lower_lo(prev), ht_upper_lo(prev)) :
      ((0.0 - 1.0), 1.0, ht_lower_both(prev), ht_upper_both(prev))))));
```

Soundness check. Output:

```
Proved Affine
Proved HardTanh
Provesound result: None
```

`https://github.com/SiyuLi915/CS521/tree/main/hw2/q5/deeppoly_hardtanh.cf`